

FinGo SaaS

Arquitetura, Código & Engenharia

Guia técnico aprofundado cobrindo o Monólito Modular, Borda Cloudflare, APIs Serverless, Banco Serverless Neon PostgreSQL, Pipeline BIM 3D, Segurança e Mapeamento de Domínios.

VERSÃO DO SISTEMA

2.40.6 (Patch 56 Security)

CLASSIFICAÇÃO

Engenharia de Software / SaaS B2B

DOMÍNIO PRINCIPAL

fingo.api.br

DATA DE EMISSÃO

Outubro / 2026

1. Visão Geral da Plataforma e Filosofia de Engenharia

O **FinGo** (anteriormente FinObra) é uma plataforma SaaS B2B de ponta desenvolvida especificamente para o ecossistema da construção civil brasileira: construtoras, incorporadoras, empreiteiras, engenheiros civis e arquitetos.

O sistema foi projetado para unificar sob uma única experiência de usuário controles que tradicionalmente operavam fragmentados em dezenas de planilhas e softwares legados: fluxo de caixa multitenant, orçamentação analítica conectada à base oficial da Caixa Econômica Federal (**SINAPI**), medições de contratos com retenções tributárias na fonte (ISS, INSS, IRRF), monitoramento automático de notas fiscais SEFAZ via Certificado A1, automação de canteiro de obras via WhatsApp e visualizador 3D BIM com clash detection em tempo real.

1.1 Princípios de Design de Software

- **Monólito Modular com Fronteiras Rígidas:** Separação estrita em Bounded Contexts (Fiscal, Financeiro, Obras, Suprimentos, Contratos, Atendimento, Gestão e Configurações). Elimina o custo de orquestração de microsserviços sem gerar acoplamento espaguete.
- **Processamento de Borda (Edge-First):** A maior parte das validações, segurança ativa (WAF, Fail2Ban, Anti-Abuso), roteamento e entrega de assets estáticos ocorre na rede global do **Cloudflare Workers & Pages** com latência sub-20ms.
- **Offline-First no Canteiro:** Arquitetura de persistência local baseada em IndexedDB nativo do navegador, com barramento de sincronização idempotente com circuit-breaker e detecção de conflitos de versão.
- **Zero Framework Lock-in no Core:** O frontend principal foi desenvolvido em JavaScript Moderno ES6+ puro (Vanilla Modular), garantindo carregamento instantâneo (Time-to-Interactive < 800ms) mesmo em dispositivos móveis conectados em redes 3G/4G no canteiro.

1.2 Topologia Completa da Infraestrutura

```
NUVEM FINGO | [ CLIENTES / DISPOSITIVOS ] | Desktop / Notebook (Chrome, Edge, Firefox, Safari) | Tablets & Celulares de Canteiro (PWA com IndexedDB Local Cache) | Terceiros & Fornecedores (Portais Públicos de Medição, Assinatura e BDI) | HTTPS / WSS / TLS 1.3 | BORDA GLOBAL CLOUDFLARE (Edge Worker & Pages) | fingo.api.br | Reverse Proxy & SPA Fallback (/app, /bim, /master, /portal) | CDN Global de Distribuição Sanitizada (pasta dist/ gerada por vite) | WAF & Fail2Ban Distribuído no Cloudflare KV (Ban de Scanners e IPs de Força Bruta) | Reputação de Segurança & Bloqueio Imediato de Scanners (.env, .aws, .git) | Storage de Arquivos & Documentos de Obra no Cloudflare R2 (fingo-attachments) | Conector MCP (Model Context Protocol) & AI Agent Cards (RFC 9727 / RFC 9116) | MICRO-APIS SERVERLESS (api/*) | CONTAINER 24/7 RENDER (backend/server.js) | Multiplexação em 12 rotas públicas | Conexão contínua WhatsApp Baileys | Autenticação JWT / TOTP RFC 6238 | Robô SINAPI 27 UFs (Caixa/IBGE) | CRUD financeiro, medições e obras | Cron jobs periódicos e DF-e SEFAZ | Conexão pooling direta Neon | Docker alpine leve com healthcheck | BANCO DE DADOS NEON POSTGRESQL (Lakebase Serverless) | Multi-Tenancy obrigatório com FORCE ROW LEVEL SECURITY (RLS) | Role finobra_app (NOBYPASSRLS) para operações de runtime isoladas | Role neondb_owner restrita exclusivamente para migrações DDL | Versionamento atômico em migrations/ (001 a 036+)
```

2. Mapeamento de Domínios e Estrutura do Código

A arquitetura do FinGo organiza o código em 8 domínios funcionais espelhados de ponta a ponta entre Frontend e Backend, permitindo que alterações de regras de negócio fiquem estritamente isoladas em seus respectivos submódulos.

2.1 Árvore de Diretórios do Repositório

```
d:/Projects/FINANÇAS/
├── api/
│   ├── _auth.js
│   ├── _database.js
│   ├── _edge-security.js
│   ├── _edge-ledger.js
│   ├── _sefaz-dfe.js
│   ├── _workflow.js
│   ├── auth.js
│   ├── db.js
│   ├── dashboard.js
│   └── whatsapp.js
├── backend/
│   ├── server.js
│   ├── sinapi_robot.js
│   └── domains/
├── frontend/
│   ├── core/
│   ├── domains/
│   ├── app.html
│   ├── master.html
│   └── bim.html
├── cloudflare-worker.js
├── migrations/
├── scripts/
└── dist/

# Handlers Serverless Node.js (Vercel & Edge Adapter)
# Módulo criptográfico de senhas (scrypt) e tokens JWT
# Gerenciador de conexão pooling com Neon Postgres
# WAF de borda, Fail2Ban, reputação de IPs e scanners
# Trilha de auditoria criptográfica SHA-256
# Integração mTLS com SEFAZ via Certificado A1
# Motor de cálculo e cascata de etapas e SLAs
# Endpoint público /api/auth
# Endpoint público /api/db (Delta Sync e Mutações)
# Endpoint público /api/dashboard (KPIs)
# Endpoint público /api/whatsapp
# Servidor Contínuo 24/7 (Container Render)
# Ponto de entrada Express, Baileys e Cron
# Robô autônomo de download e parsing da base SINAPI
# Espelho modular backend por Bounded Contexts
# Código-Fonte do Cliente Web SPA
# auth.js, data.js (IndexedDB), ui.js, events.js
# Módulos de domínio (fiscal, financeiro, obras, etc.)
# Shell da aplicação logada
# Shell de governança do superadministrador
# Shell do visualizador 3D BIM
# Gateway Edge reverso no Cloudflare Workers
# Scripts SQL atômicos de evolução de schema (001 a 036)
# Ferramentas de CI/CD, testes rápidos e deploys
# Artefatos sanitizados de produção para Cloudflare Pages
```

2.2 Dicionário dos 8 Bounded Contexts

Domínio	Responsabilidade Central	Componentes e Tabelas Chave
1. Fiscal	Download, parser XML e manifesto de notas fiscais SEFAZ contra o CNPJ da construtora usando certificado A1.	nfe.js , _sefaz-dfe.js , _certificado.js Tabelas: tenant_certificates , documentos
2. Financeiro	Contas a pagar/receber, extratos bancários, conciliação OFX inteligente, DRE Gerencial e checkout PIX.	lancamentos.js , contas.js , ofx.js , _plans.js Tabelas: lancamentos , contas , billing_invoices
3. Obras & Engenharia	Controle de canteiro, orçamentos SINAPI desonerados, boletins de medição com retenções e visualizador BIM.	obra_detalhe.js , medicoes.js , bim_viewer.js Tabelas: obras , medicoes , workflow_etapas
4. Suprimentos	Requisição de insumos direto do canteiro (pré-compras), cotações e catálogo de fornecedores.	precompras.js , produtos.js , fornecedores.js Tabelas: precompras , fornecedores , produtos
5. Contratos	Geração de instrumentos jurídicos, assinatura eletrônica com carimbo de tempo e hash SHA-256 (ICP-Brasil).	contratos.js , assinador.js , gdrive.js Tabelas: contratos , assinaturas
6. Atendimento	Comunicação ativa por WhatsApp, avisos de cobrança, alerta de etapas e assistente FinBot.	whatsapp.js , suporte.js , notificacoes.js Integração: Baileys WebSocket API / Render
7. Gestão Executiva	Dashboard em tempo real de liquidez, inadimplência, central do gestor, Kanban e Gantt.	dashboard.js , central_gestor.js , minhas_demandas.js Cálculos analíticos em tempo real
8. Configurações	Administração de usuários, cargos, políticas de senhas, auditoria criptográfica e multi-tenancy.	configuracoes.js , master.js , dev-tenant-keys.js Tabelas: tenants , usuarios , audit_ledger

3. Segurança Cibernética, Criptografia e Proteção de Borda

A segurança no FinGo é estruturada segundo o princípio militar de **Defesa em Profundidade (*Defense in Depth*)**, com barreiras rigorosas na borda DNS, no gateway de requisições, no runtime de autenticação e no isolamento físico do banco de dados.

3.1 Camada 1: WAF de Borda, Fail2Ban e Reputação de IPs

O arquivo `api/_edge-security.js` executa no Cloudflare Worker antes de qualquer processamento de rota:

- **Reputação de IPs Maliciosos (Bélgica e EUA):** Bloqueio imediato com `403 Forbidden (EDGE_IP_BANNED)` de faixas e endereços de scanners agressivos conhecidos (como `34.38.113.44` da GCP Bélgica e `40.160.65.14` da Microsoft Azure).
- **Bloqueio de Probes de Vulnerabilidades:** Intercepção sumária de requisições para arquivos de credenciais e configurações expostas:
 - Variantes de `.env`: `/.env.old`, `/.env.staging`, `/api/.env`, `/config/.env`.
 - Configurações de nuvem: `/.aws/config`, `/.aws/credentials`, `/.docker/config.json`.
 - Controle de versão e dotfiles: `/.git/config`, `/.vscode/settings.json`, `/.ssh/id_rsa`.
 - Scripts e dumps perigosos: `wp-config.php`, `dump.sql`, `backup.sql`, `phpmyadmin`.
- **Bloqueio de User-Agents Ofensivos:** Detecção e corte de tráfego de bots ofensivos (`AgenstryBot`, `cloud-crawler`, `nikto`, `sqlmap`, `censys`, `shodan`).
- **Fail2Ban Distribuído no Cloudflare KV:** Qualquer IP que atinja 5 falhas de autenticação ou comportamento anômalo em uma janela de 10 minutos é banido automaticamente por 30 minutos em toda a rede global de borda (HTTP 429).
- **Idempotência Criptográfica:** Header `X-Idempotency-Key` em requisições mutantes (POST/PUT) previne cliques duplos e duplicidade de cobranças ou lançamentos bancários (HTTP 409).
- **Conformidade RFC 9116:** Arquivo público padronizado servido na borda em `https://fingo.api.br/.well-known/security.txt`.

3.2 Camada 2: Política Rigorosa de Senhas e Hashing Assíncrono com Scrypt

Toda a gestão de credenciais no FinGo cumpre os padrões mais rigorosos do NIST SP 800-63B e da OWASP:

- **Requisitos de Complexidade de Senha:**
 - Mínimo de 8 caracteres e máximo de 128 caracteres.
 - Pelo menos 1 letra maiúscula e 1 letra minúscula.
 - Pelo menos 1 número e proibição expressa de sequências numéricas óbvias (`123456`, `654321`, `012345`).
 - Pelo menos 1 símbolo ou caractere especial (`!@#$%^&*()_+=[{}|;:,.<>?`).
- **Derivação de Chave Criptográfica com `crypto.scrypt`:** A derivação é executada de forma assíncrona com alto consumo de memória e CPU (parâmetros de custo: `N=16384`, `r=8`, `p=1`, 64 bytes de saída) acoplada a um `Salt` criptograficamente aleatório de 16 bytes gerado por `crypto.randomBytes(16)`. O formato persistido é `scrypt:N:r:p:saltHex:hashHex`, tornando ataques de dicionário e tabelas rainbow computacionalmente inviáveis mesmo em caso de vazamento.

- **Autenticação de Dois Fatores (2FA / TOTP):** Compatível com Google Authenticator e 1Password conforme a RFC 6238, com geração de códigos de contingência unívocos.

3.3 Camada 3: Content Security Policy (CSP) Estrita e Event Bridge

Para anular vetores de Cross-Site Scripting (XSS), o FinGo opera sob a diretiva de cabeçalho `script-src-attr 'none'`. Nenhum evento inline (como `onclick` ou `onload`) é permitido nos templates HTML. Em vez disso, o barramento declarativo em `js/patch26-events.js` escuta atributos desacoplados como `data-fb-click="salvar_lancamento"`, associando-os com segurança no ciclo de vida da aplicação.

4. Banco de Dados: Neon PostgreSQL & Multi-Tenancy

O banco de dados do FinGo é hospedado na arquitetura serverless **Neon Lakebase Postgres**, beneficiando-se de auto-scaling, scale-to-zero para redução de custos, pooling de conexões transacionais e alta resiliência de dados.

4.1 Isolamento Rígido com Row Level Security (RLS)

O multi-tenancy é assegurado a nível de banco de dados por políticas ativas de RLS, garantindo que mesmo se houver uma falha de lógica na camada de aplicação, nenhuma construtora conseguirá ler ou modificar os registros de outra:

```
-- Exemplo da política ativa no banco Neon
ALTER TABLE lancamentos ENABLE ROW LEVEL SECURITY;
ALTER TABLE lancamentos FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON lancamentos
  FOR ALL
  USING (tenant_id = current_setting('app.current_tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.current_tenant_id', true)::uuid);
```

4.2 Principais Tabelas do Sistema

Tabela	Colunas Principais	Finalidade e Regras de Integridade
tenants	id, razao_social, cnpj, plano, status, created_at, storage_limit_mb	Empresas e construtoras cadastradas. Chave raiz para todas as outras entidades do sistema.
usuarios	id, tenant_id, nome, email, senha_hash, perfil, mfa_secret, ativo	Usuários operadores. senha_hash gerado com script assíncrono. Perfil RBAC (admin, gestor, financeiro, canteiro).
obras	id, tenant_id, nome, cliente_id, endereco, metragem_total, status, cub_uf	Empreendimentos e construções. Vínculo para centros de custos, etapas e cronogramas.
workflow_etapas	id, obra_id, tenant_id, nome_etapa, ordem, predecessor_id, sla_dias, status	Etapas do cronograma em cascata. Conclusão da etapa predecessora dispara sucessora.
lancamentos	id, tenant_id, obra_id, conta_id, tipo, valor, data_vencimento, data_pagamento	Entradas e saídas financeiras, conciliações bancárias, DRE e fluxo de caixa.
medicoes	id, obra_id, tenant_id, numero, periodo_inicio, periodo_fim, valor_bruto, retencoes	Boletins de medição com deduções de impostos na fonte (INSS/ISS/IRRF) e liberação de parcelas.
precompras	id, obra_id, tenant_id, solicitante, itens, status, valor_estimado	Requisições originadas do canteiro. Estados: solicitado, cotado, aprovado, comprado.
documentos	id, tenant_id, obra_id, nome_arquivo, r2_key, mime_type, tamanho_bytes	Catálogo de anexos armazenados com segurança no Cloudflare R2 com URLs assinadas.
audit_ledger	id, tenant_id, actor_id, action, target_type, block_hash, previous_hash	Trilha de auditoria criptográfica baseada em encadeamento SHA-256 à prova de adulteração.

5. Engenharia, Orçamentos SINAPI e Pipeline BIM 3D

5.1 Motor de Orçamento Base Oficial SINAPI (Caixa / IBGE)

O FinGo implementa a metodologia regulamentar federal do **Decreto nº 7.983/2013** e da **Nova Lei de Licitações (Lei nº 14.133/2021)** para estimativa de custos e composições de serviços:

- **24 Bases Regionais por UF:** Ingestão mensal automática dos relatórios da Caixa Econômica Federal para as 27 Unidades da Federação.
- **Regimes Desonerado e Não Desonerado:** Aplicação dinâmica das alíquotas de encargos sociais sobre mão de obra (com desoneração da folha de pagamento pela Lei nº 12.546/2011).
- **Cálculo Oficial de BDI Diferenciado:** Aplicação da fórmula do Acórdão 2.622/2013 do Tribunal de Contas da União (TCU), diferenciando taxas de BDI para aquisição de materiais e execução de serviços.

5.2 Pipeline BIM 3D e Coordenação de Projetos

O visualizador tridimensional em `bim.html` e `js/bim_viewer.js` opera no navegador utilizando a biblioteca **Three.js**:

- **Carregamento Otimizado de Geometrias:** Suporte a formatos abertos GLB/GLTF e tesselação paramétrica CSG (Constructive Solid Geometry) com orçamentos de memória rigorosos (< 15 MB por modelo web).
- **Clash Detection Real:** Motor em `js/bim_clash_engine.js` para detecção matemática de interferências físicas espaciais entre disciplinas (Estrutura vs. Hidráulica vs. Elétrica) calculada diretamente nas caixas de colisão delimitadoras (*Bounding Boxes* e *BVH Trees*).
- **Invariante de Engenharia:** Conforme definido na governança do repositório, geometrias geradas proceduralmente ou conceituais nunca são promovidas a BIM autoritativo para medição oficial sem chancela técnica.

6. Especificação das APIs RESTful & Protocolo MCP

Todas as requisições para a API exigem transporte seguro sob HTTPS, cabeçalhos de conteúdo JSON e identificação de tenant.

6.1 Especificação dos Principais Endpoints

Método	Endpoint	Ação / Parâmetros	Resposta Típica
POST	/api/auth?action=login	Validação de credenciais (email e senha scrypt).	{ success: true, token, user }
POST	/api/auth?action=2fa_verify	Validação de token TOTP de 6 dígitos.	{ success: true, verified: true }
GET	/api/db?resource=obras	Lista obras do tenant autenticado.	{ success: true, data: [...] }
POST	/api/db?resource=lancamentos	Criação de transação financeira com idempotência.	{ success: true, id: " ... " }
GET	/api/dashboard	Retorna KPIs agregados (saldo, DRE, prazos).	{ success: true, kpis: { ... } }
POST	/api/nfe?action=sync_sefaz	Dispara consulta DF-e SEFAZ via mTLS A1.	{ success: true, novas_notas: 3 }
POST	/api/whatsapp?action=send	Envio de mensagem de cobrança ou início de fase.	{ success: true, messageId: " ... " }
ALL	/api/mcp e /sse	Endpoint de streaming do Model Context Protocol.	Protocolo JSON-RPC 2.0 / SSE Stream

6.2 Protocolo MCP (Model Context Protocol)

O FinGo é uma das primeiras plataformas de construção civil nativamente preparada para a era de agentes de inteligência artificial. O endpoint `/api/mcp` permite que assistentes externos (Claude, Gemini, Copilot) inspecionem medições, consultem a base SINAPI e interajam com a plataforma com segurança através do protocolo formal MCP.

7. Ciclo de Vida de Deploy e Governança Operacional

7.1 Regras Inegociáveis do Repositório (AGENTS.md)

🚨 Política de Tolerância Zero para Segredos (Zero Secrets in Repository)

NUNCA salvar ou commitar credenciais reais, senhas de banco (`npg_`), connection strings completas ou tokens de API. Todo o carregamento é feito estritamente em runtime via variáveis de ambiente (`.env.local` no dev e Wrangler/Render Secrets na produção). O scanner `scripts/security-secrets-scanner.cjs` roda obrigatoriamente antes de qualquer teste.

🚨 Ordem Obrigatória: Testes → Commit → Push → Deploy

Nenhum deploy para Cloudflare ou Render pode ser executado antes que a suíte completa de testes passe (`npm run test:fast`), o commit semântico seja realizado e o push para a branch remota esteja confirmado.

7.2 Comandos Operacionais Chave

Comando	Descrição e Finalidade
<code>npm run test:fast</code>	Executa o scanner de segurança e os 118 testes automatizados em paralelo (~24s).
<code>npm run build:cloudflare</code>	Gera os artefatos de distribuição sanitizados na pasta <code>dist/</code> com vite e patch 38.
<code>node scripts/deploy-cloudflare.js</code>	Publica o Worker e as Pages no Cloudflare utilizando o token seguro do <code>.env.local</code> .
<code>node scripts/organize-modular-files.cjs</code>	Sincroniza automaticamente alterações entre a raiz pública e os domínios modulares.
<code>npm run render:deploy</code>	Dispara deploy no serviço contínuo 24/7 de WhatsApp no Render.